

บทที่ 2

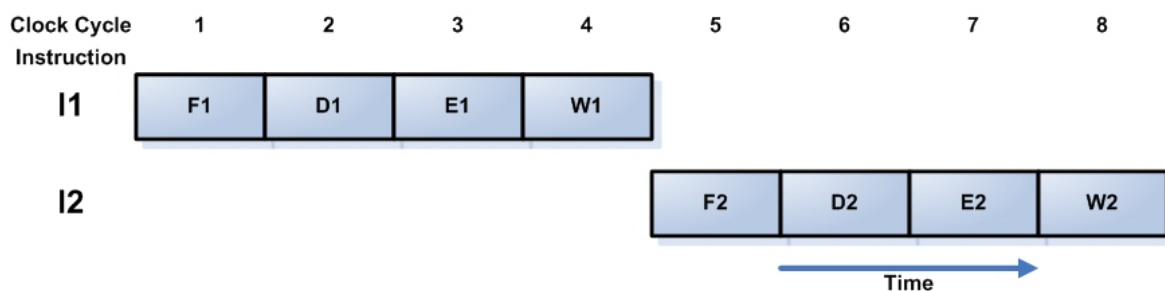
เทคนิคการทำงานแบบไปป์ไลน์

2.1 หลักการทำงานของไปป์ไลน์

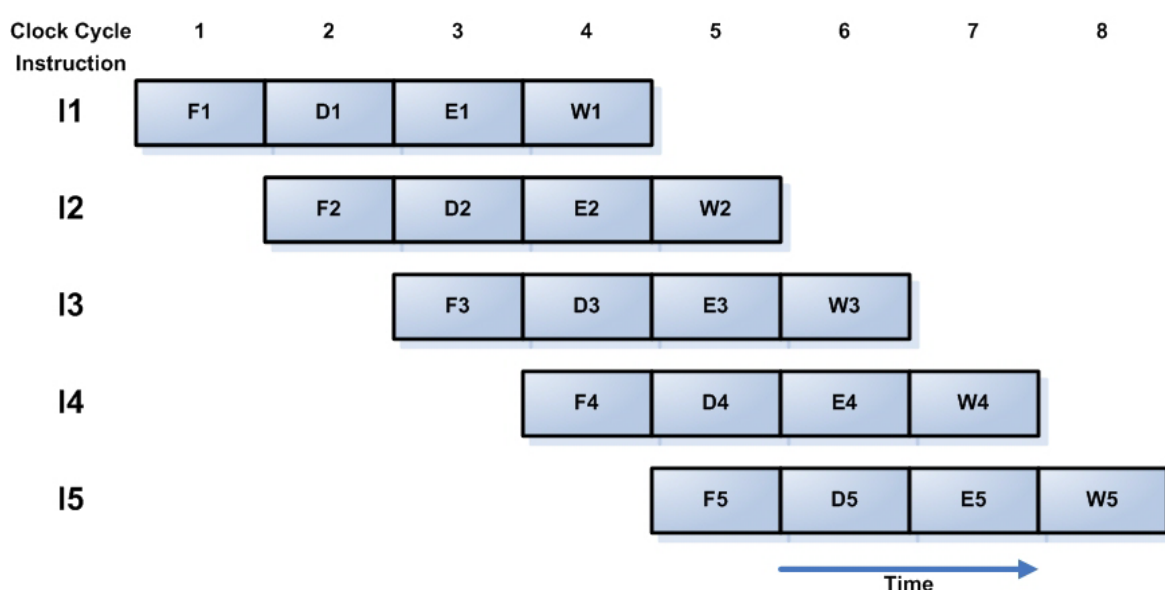
ไปป์ไลน์ (Pipelining) เป็นวิธีที่ช่วยเพิ่มความสามารถในการทำงานของซีพียู โดยการแบ่งซีพียูออกเป็นสเตจการทำงานย่อยๆ ซึ่งสามารถทำงานพร้อมกันได้ ปกติแล้วการทำงานของซีพียูนั้น จะทำงานตามคำสั่งเรียงตามลำดับ ตั้งแต่ทำการอ่านคำสั่งเข้ามา ทำการถอดรหัสคำสั่งให้เป็นคำสั่งที่แท้จริงว่าจะทำงานอะไร แล้วก็ทำการประมวลผลการทำงานของคำสั่งนั้นจนได้ผลลัพธ์ที่ต้องการ และสุดท้ายก็ทำการบันทึกผลลัพธ์การทำงานลงในหน่วยความจำหรือรีจิสเตอร์ ทำเช่นนี้ไปเรื่อยๆ กับคำสั่งต่างๆ ที่เข้ามา จะสังเกตว่า ขณะที่ทำงานคำสั่งหนึ่งนั้น ส่วนประกอบต่างๆ ภายในของซีพียูจะมีการทำงานในช่วงระยะเวลาสั้นๆ เวลาหนึ่ง หลังจากเสร็จสิ้นการทำงานแล้ว ก็จะหยุดรอการทำงานของส่วนอื่น ต่อไป หากการทำคำสั่งต่างๆ นั้นใช้เวลาในการทำงานมาก เวลาที่ส่วนประกอบต่างๆ หยุดรอก็จะมีค่ามาก เมื่อทำการพิจารณาแล้วจะเห็นได้ว่าเป็นการสิ้นเปลืองเวลาไปโดยเปล่าประโยชน์ในการทำคำสั่งคำสั่งหนึ่ง แต่หากทำไปป์ไลน์ให้มีการทำงานพร้อมกัน โดยเมื่อสเตจหนึ่งจะทำงานเสร็จในรอบการทำงานหนึ่ง จะส่งผลการทำงานของมันรวมทั้งข้อมูลที่เกี่ยวข้องให้กับสเตจต่อไป แล้วรับข้อมูลที่เกี่ยวข้องจากสเตจที่อยู่ก่อนหน้าและผลลัพธ์เข้ามาใหม่เพื่อทำการประมวลผลต่อในรอบต่อไป ในแต่ละสเตจจะสามารถทำงานพร้อมกันได้แต่ทำอยู่บนคำสั่งคนละคำสั่งกัน ดังนั้น เมื่อมองจากผลรวมของการทำงานแล้วจะให้ ผลลัพธ์การทำงาน 1 คำสั่งต่อ 1 รอบสัญญาณนาฬิกา ทำให้ได้ผลการทำงานที่รวดเร็วขึ้น ตัวอย่างเช่น หากการแบ่งการทำงานของซีพียูออกเป็น 4 สเตจ ดังนี้

1. Fetch Stage ทำการอ่านข้อมูลจากหน่วยความจำเข้ามาในซีพียู
2. Decode Stage ทำการถอดรหัสคำสั่งและเตรียมโอเพอเรนด์ให้กับการทำงานของซีพียู
3. Execute Stage ทำการประมวลผลคำสั่ง เช่น บวก ลบ เลื่อนข้อมูล เป็นต้น
4. Writeback Stage ทำการเขียนผลลัพธ์การทำงานคำสั่งลงในรีจิสเตอร์ หรือหน่วยความจำ

รูปที่ 2.1 แสดงการทำงานของซีพียูที่ไม่มีการทำไปป์ไลน์ ซึ่งการทำงานคำสั่งต่างๆ ก็จะทำตามสเตจเรียงต่อเนื่องกันไป โดยรอบสัญญาณนาฬิกาที่ใช้จะมีจำนวนมากทำให้การทำงานมีระยะเวลานาน รูปที่ 2.2 เป็นลักษณะการทำงานแบบไปป์ไลน์ขณะที่ซีพียูทำการอ่านคำสั่งที่ 5 อยู่ก็ยังทำการถอดรหัสคำสั่งที่ 4 ทำการประมวลผลคำสั่งที่ 3 และทำการเขียนผลลัพธ์ของคำสั่งที่ 2 ไปในเวลาเดียวกัน ขณะที่เขียนผลลัพธ์คำสั่งที่ 1 เสร็จสิ้นไปแล้วด้วย ดังนั้น หากสามารถทำให้ทุกคำสั่งทำงานแยกจากกันได้อย่างอิสระไม่ต้องหยุดรอกันแล้ว ซีพียูจะสามารถทำคำสั่งหนึ่งคำสั่งเสร็จภายในหนึ่งรอบสัญญาณนาฬิกาที่เร็วขึ้นได้ จะเห็นว่าเมื่อใช้การทำงานแบบไปป์ไลน์กับการออกแบบสเตจการทำงานที่เท่ากันแล้วจะสามารถทำคำสั่งได้มากกว่าในระยะเวลาที่เท่ากัน



รูปที่ 2.1 การทำงานของคำสั่งแบบธรรมดา

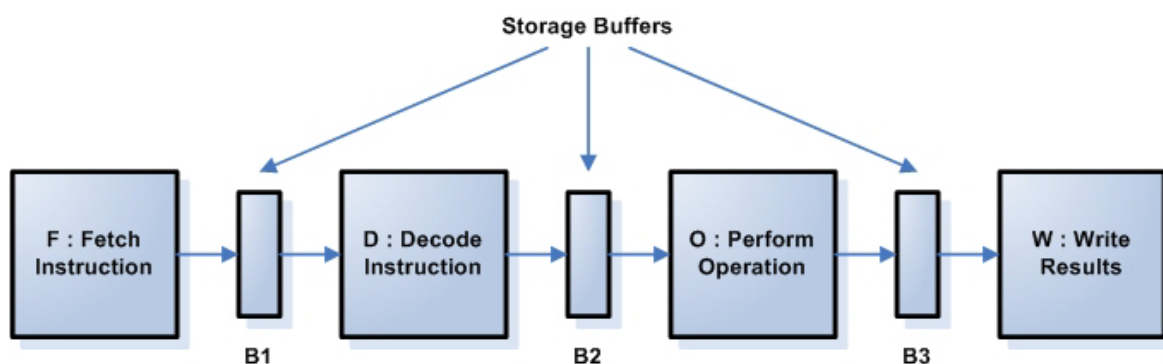


รูปที่ 2.2 การทำงานของคำสั่งเมื่อใช้เทคนิคไปป์ไลน์

หากทำการออกแบบโครงสร้างของซีพียูตามตัวอย่างข้างต้นแล้ว จะต้องทำการแบ่งฮาร์ดแวร์ออกเป็น 4 ส่วนแยกจากกันโดยในแต่ละส่วนจะต้องทำงานนานกันไปได้ ซึ่งต้องทำงานอยู่บนข้อมูลคนละชุดกันและส่งผลลัพธ์ให้กับส่วนถัดไปโดยผ่านบัฟเฟอร์เก็บข้อมูล(Storage Buffer) ถ้าหากใช้การแบ่งการทำงาน 4 สเตจแล้ว จะต้องใช้บัฟเฟอร์ 3 ตัวด้วยกันดังรูปที่ 2.3 ซึ่งเมื่อทำงานอยู่ในรอบสัญญาณนาฬิกาที่ 4 ข้อมูลในบัฟเฟอร์แต่ละตัวมีค่าดังนี้

- บัฟเฟอร์ B1 เก็บคำสั่งที่ 3 ซึ่งถูกอ่านเข้ามาในรอบสัญญาณนาฬิกาที่ 3 และกำลังถูกถอดรหัสด้วยหน่วยถอดรหัสคำสั่ง (Decode Instruction Unit)
- บัฟเฟอร์ B2 เก็บโอเปอเรนด์ของคำสั่งที่ 2 และรายละเอียดของการทำคำสั่งที่ 2 ที่ถูกถอดรหัสมาในรอบสัญญาณนาฬิกาที่ 3 สำหรับการทำการคำนวณ

- บัฟเฟอร์ B3 เก็บผลลัพธ์การทำงานที่ได้จากรอบสัญญาณนาฬิกาที่ 3 ของคำสั่งที่ 1

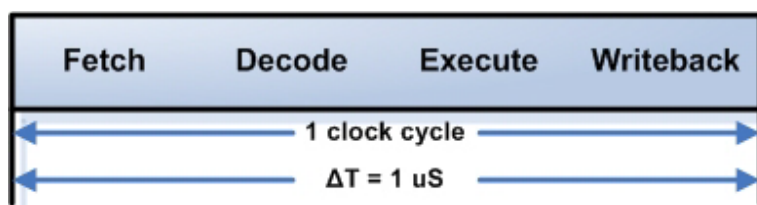


รูปที่ 2.3 โครงสร้างทางฮาร์ดแวร์ของซีพียูที่ใช้เทคนิคไปป์ไลน์

2.2 ประสิทธิภาพที่เพิ่มขึ้นจากการใช้ไปป์ไลน์

ในการเปรียบเทียบประสิทธิภาพระหว่างซีพียูที่ใช้ไปป์ไลน์กับซีพียูปกติ จะทำการพิจารณาในสองประการ ประการแรกคือการพิจารณาประสิทธิภาพที่เพิ่มขึ้นของแต่ละคำสั่ง ประการที่สองคือการพิจารณาจากประสิทธิภาพโดยรวมของระบบ จากตัวอย่างรูปที่ 2.4 เมื่อซีพียูทำงานแบบไม่ใช้ไปป์ไลน์ในขั้นตอนการทำงานแต่ละคำสั่งจะต้องทำทั้ง 4 ขั้นตอน หากสามารถทำงานแต่ละขั้นตอนเสร็จใน 1 รอบสัญญาณนาฬิกาแล้ว ในการทำงาน 1 คำสั่งจะเสร็จสิ้นโดยใช้เวลาในการทำงานเท่ากับ 1 ไมโครวินาที เมื่อซีพียูทำงานที่ความเร็ว 1 เมกะเฮิร์ตซ์ ดังนั้น สมมติว่าเวลาผ่านไป 5 วินาทีก็จะสามารถทำงานได้ 5 ล้านคำสั่ง

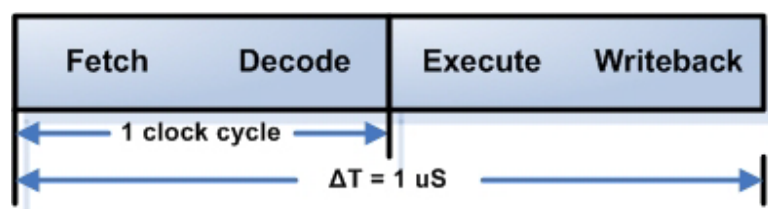
$$5 \text{ seconds} \times \left(\frac{1 \times 10^6 \text{ cycles}}{1 \text{ second}} \right) \times \left(\frac{1 \text{ instructions} \times 1 \text{ operations}}{1 \text{ cycle}} \right) = 5 \times 10^6 \text{ instructions}$$



รูปที่ 2.4 การทำงานของซีพียูที่ไม่มีการทำไปป์ไลน์

หากทำการแบ่งซีพียูในรูปที่ 2.4 ให้มีการทำงานแบบไปป์ไลน์ 2 สเตจ สเตจละ 2 ขั้นตอนโดยสเตจที่ 1 ทำการอ่านคำสั่งและถอดรหัสคำสั่ง แล้วส่งต่อไปให้สเตจที่ 2 ทำการประมวลผลและเขียนผลลัพธ์ จะทำให้เวลาในการทำงานของแต่ละขั้นตอนเหลือเพียงครึ่งเดียว ดังนั้น สามารถเพิ่มความเร็วของสัญญาณนาฬิกาขึ้นเป็น 2 เท่าได้ แต่หากพิจารณาให้ดีจะเห็นว่า แม้ความเร็วจะเพิ่มเป็น 2 เท่าแต่เวลาในแต่ละรอบสัญญาณนาฬิกานั้นสามารถทำงานได้เพียงแค่ครึ่งคำสั่งเท่านั้น แต่ละคำสั่งต้องใช้การทำงาน 2 รอบสัญญาณนาฬิกา ดังนั้น คงทำแต่ละคำสั่งโดยใช้เวลาเท่าเดิม อย่างไรก็ตามแม้ว่าคำสั่งจะใช้เวลาในการทำงานเท่าเดิมแต่ว่ามันสามารถทำงานได้ 2 คำสั่งในเวลาเดียวกัน ดังนั้นเมื่อมองค่าของคำสั่งต่อสัญญาณนาฬิกา (Instruction per Clock: IPC) แล้วจะได้ค่าเท่ากับ 2 เมื่อเวลาผ่านไป 5 วินาทีเท่ากับกับกรณีแรกจะสามารถทำคำสั่งเท่ากับ 10 ล้านคำสั่ง

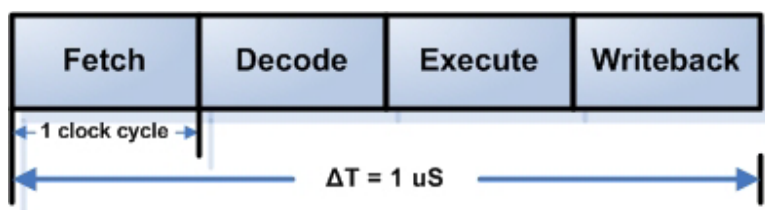
$$5 \text{ seconds} \times \left(\frac{2 \times 10^6 \text{ cycles}}{1 \text{ second}} \right) \times \left(\frac{2 \text{ instructions} \times 0.5 \text{ operations}}{1 \text{ cycle}} \right) = 10 \times 10^6 \text{ instructions}$$



รูปที่ 2.5 การทำงานของซีพียู ที่มีการทำไปป์ไลน์ 2 สเตจ

ถ้าหากทำการแบ่งซีพียูในรูปที่ 2.4 ใหม่ให้ทำงานแบบไปป์ไลน์ 4 สเตจแทนดังรูปที่ 2.6 โดยสเตจที่ 1 ทำการอ่านคำสั่ง สเตจที่ 2 ทำการถอดรหัสคำสั่ง สเตจที่ 3 ทำการประมวลผลแล้วส่งต่อไปให้สเตจสุดท้ายทำการเขียนผลลัพธ์จะทำให้เวลาในการทำงานของแต่ละขั้นตอนเหลือเพียงหนึ่งในสี่ส่วน สามารถเพิ่มความเร็วของสัญญาณนาฬิกาขึ้นเป็น 4 เท่าได้ ในทำนองเดียวกับการทำงานแบบ 2 สเตจแม้ว่าความเร็วจะเพิ่มเป็น 4 เท่าแต่เวลาในแต่ละรอบสัญญาณนาฬิกานั้นสามารถทำงานได้เพียงแค่หนึ่งในสี่ส่วนของคำสั่งเท่านั้น ดังนั้น แต่ละคำสั่งจึงต้องใช้การทำงาน 4 รอบสัญญาณนาฬิกาทำให้ยังคงใช้เวลาในการทำงานแต่ละคำสั่งเท่าเดิม เมื่อเวลาผ่านไป 5 วินาที จะเห็นว่าซีพียูดังกล่าวจะสามารถทำงานได้ถึง 20 ล้านคำสั่ง

$$5 \text{ seconds} \times \left(\frac{4 \times 10^6 \text{ cycles}}{1 \text{ second}} \right) \times \left(\frac{4 \text{ instructions} \times 0.25 \text{ operations}}{1 \text{ cycle}} \right) = 20 \times 10^6 \text{ instructions}$$



รูปที่ 2.6 การทำงานของซีพียูที่มีการทำไปป์ไลน์ 4 สเตจ

2.3 จำนวนสเตจของไปป์ไลน์

แม้ว่าในทางทฤษฎีแล้วการมีจำนวนสเตจของไปป์ไลน์มากก็ยิ่งจะทำให้ประสิทธิภาพเพิ่มมากขึ้นตามไปด้วย แต่ในการทำแต่ละสเตจของไปป์ไลน์นั้นจะมีส่วนของโอเวอร์เฮดของการเคลื่อนย้ายข้อมูลจากบัฟเฟอร์ของสเตจหนึ่งไปยังอีก สเตจหนึ่ง และการจัดเตรียมฟังก์ชันการทำงานเพื่อส่งไปยัง สเตจถัดไป ดังนั้น จำนวนสเตจที่เพิ่มมากขึ้นจะทำให้ซีพียูต้องการจำนวนเกตและทรานซิสเตอร์ในการออกแบบที่เพิ่มขึ้นใน ขณะเดียวกันจำนวนวงจรลอจิกที่ใช้ในการแก้ปัญหการขึ้นต่อกันของข้อมูลในหน่วยความจำและรีจิสเตอร์ ก็จะเพิ่มความซับซ้อนขึ้นมากเมื่อเทียบกับจำนวนสเตจที่ เพิ่มขึ้นเพียงไม่กี่สเตจ ส่งผลให้วงจรควบคุมการทำงานระหว่างสเตจอาจมีความซับซ้อนมากกว่าสเตจที่ถูกควบคุมเสียอีก นอกจากนี้แล้วการเพิ่มจำนวนสเตจยังมีผลให้การทำงานของไปป์ไลน์เกิดการติดขัด (Stall) ได้มากขึ้นเนื่องจากปัญหาในการทำงานจะเพิ่มสูงขึ้นดังนั้น ซีพียูส่วนใหญ่จึงมักมีจำนวนสเตจของไปป์ไลน์อย่างมากไม่เกิน 10 สเตจเท่านั้น

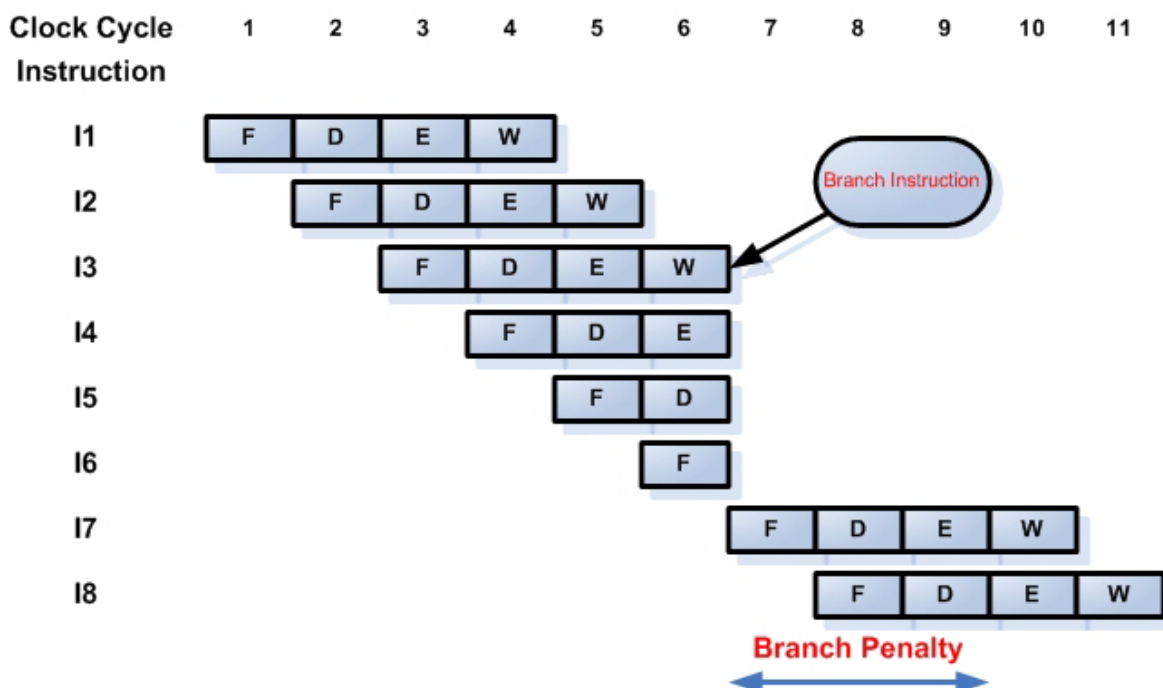
2.4 ปัญหาที่เกิดจากการใช้ไปป์ไลน์และเทคนิคในการแก้ปัญหา

ปัญหาที่เกิดขึ้นกับซีพียูที่ใช้เทคนิคไปป์ไลน์ในการทำงานที่เกิดขึ้นได้จะแบ่งออกเป็นปัญหาใหญ่ๆ 3 ด้านด้วยกัน คือ ปัญหาด้านการควบคุม (Control Hazards) ปัญหาด้านการขึ้นต่อกันของข้อมูล (Data Hazards) และปัญหาด้านโครงสร้าง (Structure Hazards)

2.4.1 ปัญหาด้านการควบคุม (Control Hazards)

ปัญหาด้านการควบคุมมักเกิดขึ้นเมื่อมีการทำคำสั่งที่เกี่ยวข้องกับการกระโดด (Branch Instruction) เช่น หากคำสั่ง I_j ที่กำลังถอดรหัสเป็นกลุ่มคำสั่งกระโดด และผลลัพธ์จากการทำคำสั่งทำให้เกิดการกระโดดไปยังตำแหน่งเป้าหมายทำให้ซีพียูต้องทิ้งการทำคำสั่งตั้งแต่คำสั่ง I_{j+1} ถึงคำสั่ง $I_{j+(n-1)}$ (เมื่อ n เป็นจำนวนสเตจของไปป์ไลน์) ซึ่งเป็นคำสั่งที่อยู่หลังคำสั่งกระโดด และเริ่มการอ่านคำสั่งที่ I_k ซึ่งเป็นคำสั่งใหม่ที่อยู่ที่ตำแหน่งเป้าหมายของการกระโดด ดังรูปที่ 2.7 จะเห็นว่าสัญญาณนาฬิกาถูกที่ 7 - 9 นั้นซีพียูไม่ได้ทำงานให้ผลลัพธ์เกิดขึ้นเลย เวลาที่เสียไปจากผลของการกระโดดในลักษณะนี้เรียกว่าผลเสียที่เกิดจากคำสั่งกระโดด (Branch Penalty)

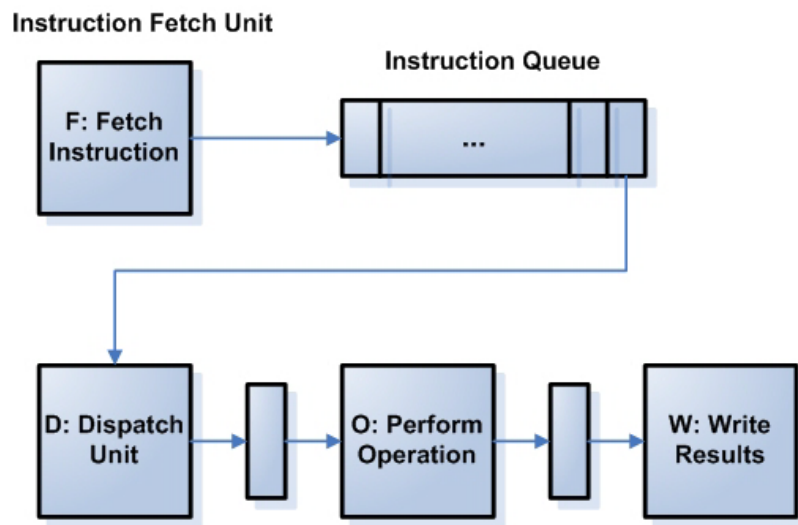
จากรูปที่ 2.7 จะเห็นว่าในขณะที่ทำการเขียนผลลัพธ์ของคำสั่งที่ 3 อยู่ นั้นซีพียูจะต้องทำการทิ้งข้อมูลของคำสั่งที่ 4 – 6 ที่ได้ทำขั้นตอนก่อนหน้านั้นไปแล้วทิ้งไปทั้งหมดแล้วเริ่มทำคำสั่งที่ 7 ซึ่งเป็นคำสั่งในตำแหน่งเป้าหมายที่กระโดดไปใหม่



รูปที่ 2.7 การทำงานของซีพียูสูญเสียไปเนื่องจากเกิดการกระโดด

ในการแก้ไขปัญหาคำสั่งกระโดดนั้นสามารถทำได้หลายวิธี ซึ่งปกติแล้วคำสั่งกระโดดแบ่งออกเป็น 2 ประเภทคือ คำสั่งกระโดดแบบมีเงื่อนไข กับคำสั่งกระโดดแบบไม่มีเงื่อนไข ซึ่งในการแก้ไขปัญหที่เกิดจากการกระโดดแบบไม่มีเงื่อนไขนั้นสามารถทำได้ โดยอาศัยการทำงานของตัวจัดลำดับคำสั่ง (Instruction Queue) เข้ามาช่วยในการเก็บคำสั่งที่ซีพียูต้องทำงานไว้ในคิวคำสั่ง แล้วให้หน่วยดึงคำสั่ง (Dispatch Unit) ดึงคำสั่งดังกล่าวออกไปใช้งานต่อไป เมื่อหน่วยอ่านคำสั่งเจอคำสั่งกระโดดแบบไม่มีเงื่อนไขแล้วมันจะอ่านคำสั่งที่อยู่ในตำแหน่งเป้าหมายของคำสั่งกระโดดมาเป็นคำสั่งถัดไป ดังรูปที่ 2.8 ส่วนการแก้ไขปัญหการกระโดดแบบมีเงื่อนไขนั้นทำได้ยากกว่ามากเนื่องจากซีพียูไม่มีทางทราบได้เลยว่ากระโดดจะเกิดขึ้นหรือไม่จนกว่าจะได้ทำคำสั่งนั้นสำเร็จแล้ว จึงได้มีความพยายามนำเสนอวิธีการในการแก้ปัญหการกระโดดแบบมีเงื่อนไขหลายวิธีดังนี้

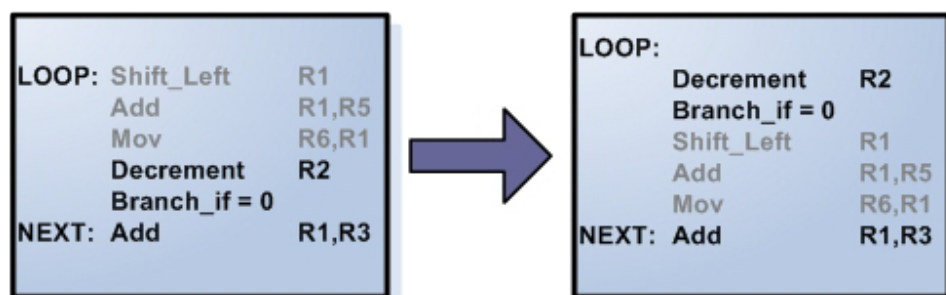
- Delayed Branch
- Multiple Streams
- Branch Prediction



รูปที่ 2.8 การใช้หน่วยจัดลำดับคำสั่งในการแก้ปัญหาคะโดดแบบไม่มีเงื่อนไข

2.4.1.1 Delayed Branch

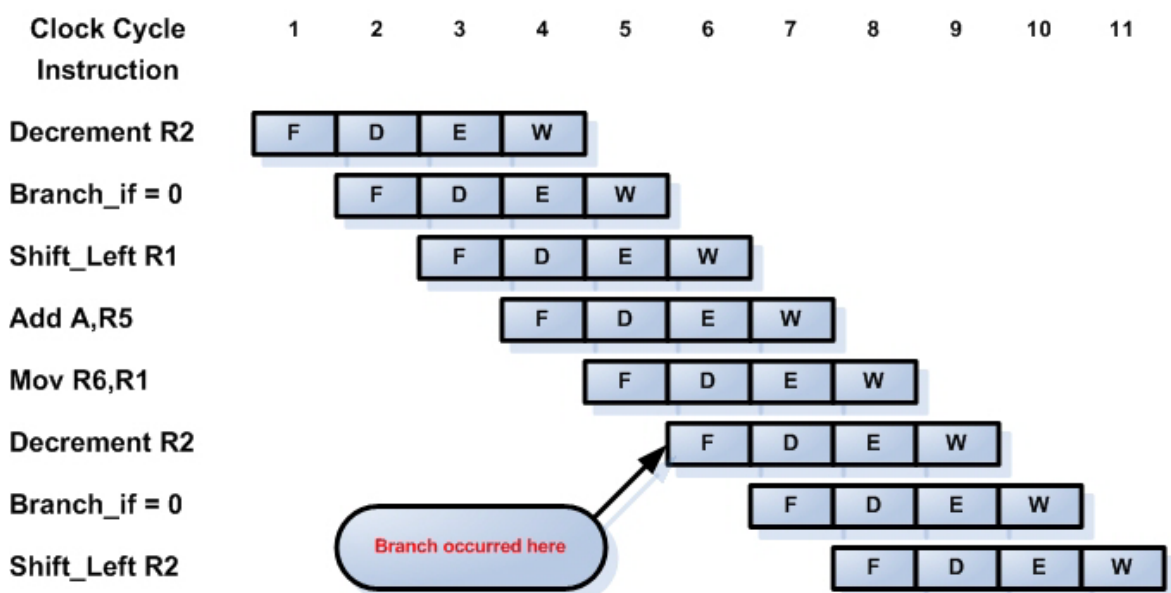
จากปัญหาของไปป์ไลน์ในรูปที่ 2.7 จะเห็นว่าซีพียูเริ่มทำคำสั่งที่ 4 – 6 ก่อนที่จะรู้ว่าคำสั่งที่ 3 นั้นเกิดการกระโดดหรือไม่ ดังนั้นเมื่อคำสั่งที่ 3 เสร็จแล้วเกิดการกระโดดขึ้น ซีพียูจะต้องทิ้งคำสั่งที่ 4 – 6 ไป ซึ่งในการแก้ไขปัญหาดังวิธีนี้จะทำการกำหนดให้ตำแหน่งที่เก็บคำสั่งหลังคำสั่งกระโดดนี้เป็น Branch Delay Slot และให้ซีพียูทำคำสั่งที่อยู่ในตำแหน่งนี้ก่อนที่จะมีการกระโดดขึ้นจริง โดยให้การกระโดดนั้นถูกยึดเวลาออกไป ดังนั้น ในการนำไปใช้งานต้องมีการเรียงคำสั่งภายในโปรแกรมใหม่แล้วจัดคำสั่งที่เหมาะสมใส่ลงไปในส่วนของ Branch Delay Slot หากไม่สามารถหาคำสั่งมาใส่ได้แล้วอาจต้องถูกเติมด้วยคำสั่ง NOP (No Operation) รูปที่ 2.9 แสดงถึงตัวอย่างส่วนของของโปรแกรมทำงานที่ใช้เทคนิค Delayed Branch นี้



รูปที่ 2.9 การแก้ไขโปรแกรมเพื่อรองรับการทำงานของ Branch Delay Slot

จากรูปที่ 2.9 จะเห็นว่าคำสั่งของโปรแกรมทางด้านซ้ายมือคำสั่งที่ 2 – 4 นั้นสามารถนำมาใส่ใน Branch Delay Slot ได้โดยไม่ทำให้การทำงานของโปรแกรมเสียไป ดังนั้นโปรแกรมที่แก้ไขแล้วทางด้านขวามือยังคงทำงานถูกต้องเหมือนเดิมทุกประการ ซึ่งการทำงานของโปรแกรมในไปป์ไลน์จะแสดงดังรูปที่ 2.10

เทคนิค Delayed Branch แม้ว่าจะสามารถนำมาใช้แก้ปัญหาได้แต่ในทางปฏิบัติประสิทธิภาพจะขึ้นอยู่ กับความสามารถของคอมไพเลอร์ ที่จะต้องหาคำสั่งที่เหมาะสมสำหรับใส่ใน Branch Delay Slot ให้ได้ ซึ่งจากการวิจัยที่ผ่านมาพบว่าเทคนิคในการคอมไพล์สมัยใหม่สามารถหาคำสั่งใส่ในสล็อตได้ประมาณร้อยละ 85 เมื่อ ขนาดของสล็อตเท่ากับ 1 สล็อต แต่ถ้าเพิ่มขนาดของสล็อตให้มากขึ้น โอกาสที่จะหาคำสั่งที่เหมาะสมมาใส่ในสล็อต จะลดลงมากทำให้ประสิทธิภาพที่ได้ต่ำ ดังนั้น หากซีพียูมีจำนวนของสเตจในการทำไปป์ไลน์มากแล้วโอกาสที่จะหาคำสั่งมาเติมลงในสล็อตได้ก็จะน้อย



รูปที่ 2.10 การทำงานของ Branch Delay Slot

2.4.1.2 Multiple Streams

เมื่อไปป์ไลน์ได้รับผลกระทบจากคำสั่งกระโดด เนื่องจากจะต้องเลือกที่จะอ่านคำสั่งใดเป็นคำสั่งถัดไป เพื่อเป็นการป้องกันความผิดพลาดของการเลือกคำสั่ง การทำ Multiple Streams นี้จะเป็นการสร้างส่วนเริ่มต้นของไปป์ไลน์ขึ้นมาอีกชุดหนึ่ง และให้ทั้งสองชุดนี้ทำการอ่านคำสั่งทั้งสองคำสั่งขึ้นมาพร้อมกัน หรือการทำให้มีสตรีมคำสั่งขึ้นมา 2 สตรีมนั่นเอง วิธีการนี้มีปัญหาในการสร้างอยู่ 2 ประการด้วยกัน ปัญหาประการแรกคือเมื่อมีหลายไปป์ไลน์สตรีม จะทำให้เกิดการแย่งกันเข้าถึงหน่วยความจำและรีจิสเตอร์ ทำให้มีผลเป็นปัญหาด้านโครงสร้างร่วมด้วย ปัญหาประการที่สองคืออาจพบคำสั่งกระโดดอื่น ๆ อีกในขณะที่กำลังทำงานเป็นสตรีมอยู่ก่อนที่จะทำคำสั่ง

กระโดดเดิมเสร็จ ซึ่งจะต้องมีสตรีมเพิ่มขึ้นมาอีกเพื่อรองรับการจัดการคำสั่งกระโดดที่อาจเกิดขึ้นใหม่ระหว่างนี้ ด้วยเป็นการเพิ่มความซับซ้อนมากยิ่งขึ้น ทำให้จัดการได้ยาก

2.4.1.3 Branch Prediction

วิธีการนี้จะใช้หลักการในการทำนายการกระโดดจากผลของคำสั่งกระโดดซึ่งมีอยู่หลายแบบที่นำมาใช้ในการทำนาย ได้แก่

- ทำนายว่าไม่เกิดการกระโดดขึ้นเลย
- ทำนายว่าเกิดการกระโดดขึ้นทุกครั้ง
- ทำนายจากการฮอปโค้ดของแต่ละคำสั่ง
- ใช้ Taken / Not Taken Switch
- ใช้ Branch History Table

แบบแรกนั้นจะทำนายว่าไม่เกิดการกระโดดขึ้นเลยซีพียูจะอ่านคำสั่งถัดไปจากการกระโดดเข้ามาทำงานเสมอ ซึ่งหากทำนายผิดพลาดก็จะเสียเวลาและเสียคำสั่งที่อ่านมาไป

แบบที่สองนี้ตรงข้ามกับแบบแรกโดยการทำการอ่านคำสั่งที่อยู่ปลายทางที่มีการกระโดดไปมาทำงานเสมอ จากการศึกษาพบว่าการทำคำสั่งกระโดดแบบมีเงื่อนไขมากกว่าร้อยละ 50 จะเกิดการกระโดด แบบนี้จึงให้ผลของการทำงานที่ดีกว่าแบบแรก แต่การนำเอาไปใช้กับซีพียูที่ใช้การทำงานกับหน่วยความจำแบบเพจ(Paging) จะทำให้เกิด Page Fault สูงขึ้นได้ ต้องมีการสร้างกลไกในการป้องกันปัญหานี้ด้วย

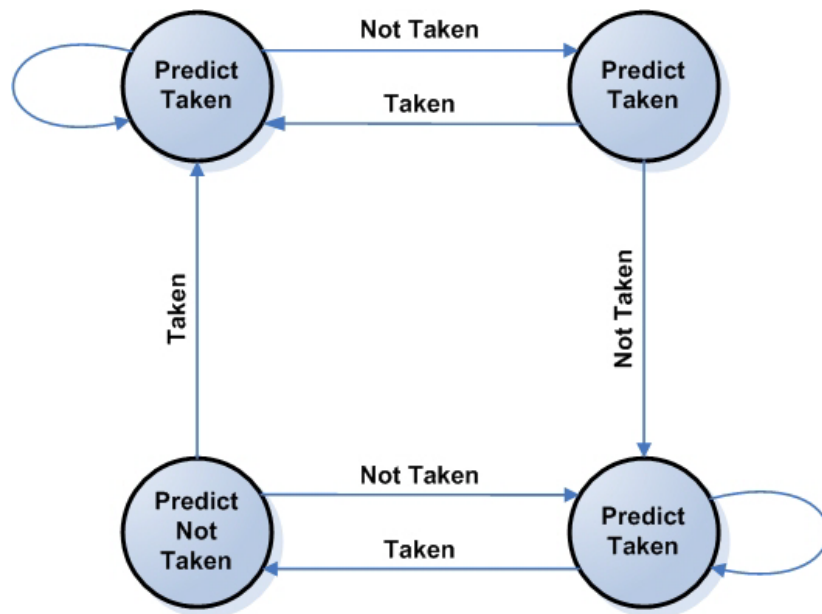
แบบที่สามทำนายโดยดูฮอปโค้ดของคำสั่ง โดยจะพิจารณาการของคำสั่งอยู่ว่าคำสั่งใดน่าจะเกิดการกระโดดขึ้นและคำสั่งใดไม่น่าจะเกิดการกระโดดขึ้นแล้วทำการทำนายไปตามรายการของคำสั่งนั้น ซึ่งสามารถให้ผลการทำนายถูกต้องอยู่ที่ประมาณร้อยละ 75

การทำนายทั้ง 3 แบบแรกนี้เป็นการทำนายแบบสเตติก คือมีค่าการทำนายคงที่ ตรงไปตรงมาไม่เปลี่ยนแปลง และไม่มีการเก็บประวัติของการทำนายไว้ในซีพียูแต่อย่างใด แบบที่จะกล่าวถึงต่อไปนั้นเป็นการทำนายแบบไดนามิกโดยมีการเก็บประวัติของการเกิดและการทำนายไว้ เพื่อใช้ในการตัดสินใจในการกระโดดที่จะเกิดขึ้นในครั้งต่อไป

แบบที่สี่ Taken / Not Taken Switch นั้นจะมีการบันทึกว่าการกระโดดครั้งล่าสุดเกิดขึ้นหรือไม่และค่านี้จะถูกนำมาใช้ในการตัดสินใจการกระโดดครั้งต่อไป โดยเก็บเป็นบิตไว้ในตัวซีพียูอาจใช้เป็นตารางขนาดเล็กในการเก็บคำสั่งที่เพิ่งทำเสร็จไป 1 บิตต่อ 1 คำสั่ง การเก็บค่าด้วยวิธีนี้สามารถเก็บได้ว่าการทำงานครั้งสุดท้ายของคำสั่งกระโดดนั้นมีการกระโดดหรือไม่ ซึ่งจะมีผลดีเมื่อทำคำสั่งที่มีการวนลูปบ่อยๆ เท่านั้นแต่จะมีข้อผิดพลาดในการทำนายเกิดขึ้น 2 ครั้งเมื่อมีการทำลูปคือการเข้าลูปและการออกจากลูปซึ่งสามารถแก้ไขโดยการเก็บค่าไว้จำนวน 2 บิต ดังรูปที่ 2.11 โดยที่กระบวนการในการตัดสินใจจะแสดงดังไฟไนต์สเตตแมชีนซึ่งมีการทำงาน 4 สเตจ หากทิศทางของการกระโดดเกิดในทิศทางเดียวกัน 2 ครั้งการทำนายครั้งต่อไปจะถือว่าเกิดในทิศทางนั้น หากการทำนายเกิดผิดพลาดขึ้นจะทำนายเหมือนเดิมอีกครั้ง แล้วถ้าหากยังผิดพลาดอีกครั้งก็จะทำนายในทิศทางตรงกันข้าม

แทน ดังนั้น การเปลี่ยนทิศทางของการทำนายจะเกิดขึ้นเมื่อมีการทำนายผิดพลาดติดต่อกันสองครั้งเท่านั้น ดังนั้น เมื่อนำมาทำลูปแล้ว การทำนายจะเกิดความผิดพลาดเพียงครั้งเดียวเท่านั้น

แบบที่ห้า คือการใช้ Branch History Table เป็นการใช้หน่วยความจำแบบแชนขนาดเล็กควบคู่ไปกับสแต็กการอ่านคำสั่งของไปป์ไลน์โดยในตารางจะประกอบด้วยสมาชิกได้แก่ History Bit เก็บสแต็กการใช้งานของคำสั่งกระโดดนั้นๆ

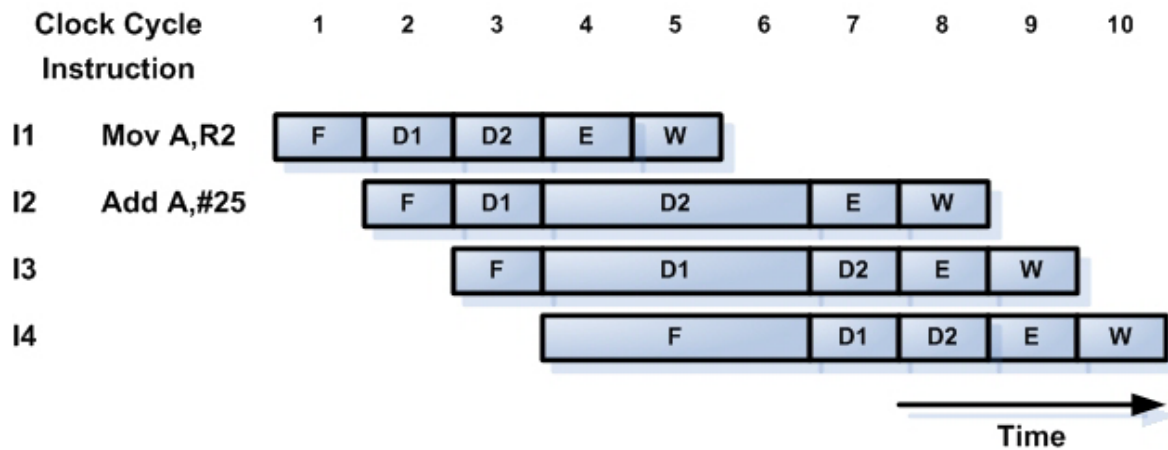


รูปที่ 2.11 แสดงไดอะแกรมในการทำนายการกระโดด

2.4.2 ปัญหาด้านการขึ้นต่อกันของข้อมูล (Data Hazards)

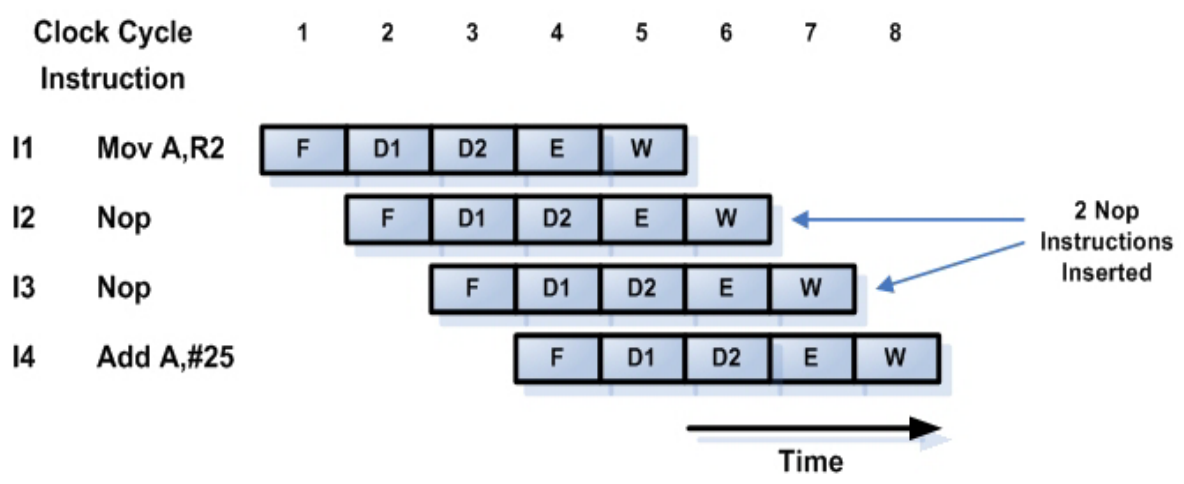
ปัญหานี้เกิดจากการที่คำสั่ง I_j ต้องใช้ผลลัพธ์ที่เกิดขึ้นจากการทำคำสั่ง I_{j-1} ที่เป็นคำสั่งก่อนหน้าทำให้ต้องรอการทำงานของคำสั่ง I_{j-1} ให้เสร็จสิ้นการเขียนผลลัพธ์เสียก่อนแล้วจึงทำการอ่านค่าของข้อมูลดังกล่าวมาใช้งานได้ ดังรูปที่ 2.12 คำสั่งที่ 2 ต้องการผลลัพธ์ของการทำคำสั่งที่ 1 ก่อนหน้าจึงต้องหยุดรอการทำงานนั้นให้เสร็จสิ้นก่อน ทำให้บางสแต็กของไปป์ไลน์เกิดการว่างงานขึ้นส่งผลให้ประสิทธิภาพของไปป์ไลน์ลดลง ปัญหาการขึ้นต่อกันของข้อมูลนี้แบ่งเป็น 3 รูปแบบได้แก่

1. การอ่านก่อนแล้วเขียนทีหลัง (Read After Write: RAW)
2. การเขียนก่อนแล้วเขียนทีหลัง (Write After Write: WAW)
3. การเขียนก่อนแล้วอ่านทีหลัง (Write After Read: WAR)

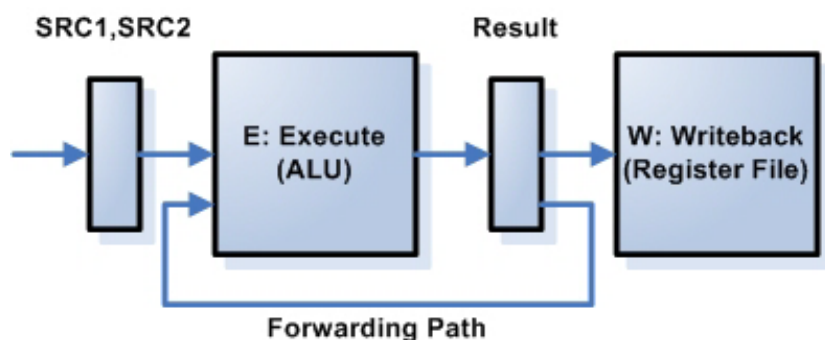


รูปที่ 2.12 การหยุดรอเมื่อเกิดการขึ้นต่อกันของข้อมูล

ปัญหา RAW เกิดจากการที่คำสั่งที่ I_{j+1} พยายามที่จะอ่านข้อมูลก่อนที่คำสั่งที่ I_j จะเขียนข้อมูลเสร็จ ดังนั้นคำสั่งที่ I_{j+1} จะได้ข้อมูลเก่าไปทำงานก่อนที่คำสั่งที่ I_j จะเขียนข้อมูลใหม่ลงไปทำให้การทำงานของโปรแกรมมีข้อผิดพลาดเกิดขึ้น ปัญหานี้สามารถแก้ไขได้ 2 วิธีคือ วิธีแรกคือการเติมคำสั่ง NOP โดยอัตโนมัติระหว่างคำสั่งที่มีการขึ้นต่อกันของข้อมูลดังรูปที่ 2.13 หรือวิธีที่สองใช้การฟอร์เวิร์ดโอเปอเรนด์ ดังรูปที่ 2.14



รูปที่ 2.13 การแก้ปัญหา RAW ด้วยการเติมคำสั่ง NOP



รูปที่ 2.14 การแก้ปัญหา RAW ด้วยการทำฟอร์เวิร์ดโอเปอเรนด์

ในการทำการฟอร์เวิร์ดโอเปอเรนด์เพื่อแก้ไขปัญห RAW นั้นสิ่งที่จะต้องคำนึงถึงคือความซับซ้อนของวงจรที่ใช้ทำการฟอร์เวิร์ดข้อมูลจากสแตจหนึ่งไปยังอีกสแตจหนึ่ง หากมีจำนวนสแตจการทำไปป์ไลน์มากวงจรที่คอยสนับสนุนการทำงานนี้ก็จะมีความซับซ้อนมากขึ้นตามไปด้วย สมมติว่าเป็นการฟอร์เวิร์ดของไปป์ไลน์จำนวน 3 สแตจ วงจรการฟอร์เวิร์ดในสแตจที่ 2 จะมีความซับซ้อนมากกว่าวงจรในสแตจที่ 1 เนื่องจากผลการฟอร์เวิร์ดจากสแตจที่ 3 จะมีผลต่อวงจรในสแตจที่ 2 และ 1 แต่ผลจากการฟอร์เวิร์ดในสแตจที่ 2 จะมีผลต่อสแตจที่ 1 เท่านั้น

ปัญหาต่อมาคือ WAW เกิดจากการที่คำสั่ง I_{j+1} เขียนข้อมูลลงไปก่อนที่คำสั่ง I_j จะทำการเขียนข้อมูลเสร็จ ทำให้ค่าของข้อมูลที่ได้เป็นค่าที่ได้จากคำสั่ง I_{j+1} แทนที่จะเป็น I_j ทำให้ผลลัพธ์ของ I_j ที่ถูกหายไปเกิดเป็น ความผิดพลาดของโปรแกรมขึ้น การป้องกันปัญหาและแก้ปัญหการเกิด WAW นี้ต้องทำการควบคุมไปป์ไลน์ซึ่งซับซ้อนมาก โดยปัญหานี้มักเกิดกับซีพียูที่ทำงานแบบไม่เรียงลำดับ (Out-of-Order Execution)

ปัญหาสุดท้ายคือการเกิด WAR จากการที่คำสั่ง I_{j+1} เขียนข้อมูลลงไปก่อนที่คำสั่ง I_j จะอ่านข้อมูลขึ้นมาทำให้คำสั่ง I_j ได้รับข้อมูลที่ผิดพลาดไปทำงานแล้วได้ผลการทำงานผิดพลาดขึ้นมา การป้องกันปัญหาและแก้ปัญหการเกิด WAR ต้องการการควบคุมไปป์ไลน์ที่ซับซ้อนเช่นกัน

2.4.3 ปัญหาด้านโครงสร้าง (Structure Hazards)

แม้ว่าในทางหลักการแล้วไปป์ไลน์จะช่วยเพิ่มความเร็วในการทำงาน โดยการทำหลายๆคำสั่งพร้อมๆกัน แต่บนงานที่แตกต่างกันแล้ว การในไปป์ไลน์สร้างวงจรใช้งานจริงกลับเกิดปัญหาในหลายด้านอันเนื่องมาจากอุปกรณ์หลายตัวไม่สนับสนุนการทำงานแบบขนาน เช่น หน่วยความจำ ในการทำงานแบบไปป์ไลน์ผู้ออกแบบต้องการให้สแตจอ่านคำสั่งสามารถอ่านหน่วยความจำ ที่เก็บคำสั่ง ในขณะที่สแตจอื่นต้องการอ่านค่าจากหน่วยความจำซึ่งอีกคำสั่งต้องการใช้งาน นอกจากนี้สแตจเขียน ผลลัพธ์ก็ต้องการเขียนผลลัพธ์ในการทำงานเช่นกัน จึงมีสแตจทำงานเกี่ยวข้องกับหน่วยความจำในเวลาเดียวกันหลายสแตจ ซึ่งหน่วยความจำนั้นยังไม่สนับสนุนในการให้อุปกรณ์หลายๆ ตัวสามารถเข้าถึงข้อมูลพร้อมกันได้จึงเกิดเป็นปัญหาด้านโครงสร้างขึ้น

การแก้ปัญหานี้สามารถทำได้หลายวิธี เช่น การ Stalling หรือ bubble เป็นการหยุดการทำงานของไปป์ไลน์เพื่อให้การทำงานในส่วนนั้นเสร็จสิ้นไปก่อนแล้วจึงอนุญาตให้ทำส่วนต่อไปได้ ตัวอย่างเช่น การทำงานของ

วงจรคูณ/หาร ขนาดใหญ่ จะใช้วิธีในการแก้ปัญหา โดยจะ stall วงจรส่วนที่อยู่ก่อนหน้า แล้วทำ การคูณ/หาร ที่ execute stage ให้เสร็จจึงทำงานตามปกติต่อไป หรือในส่วนของหน่วยความจำข้อมูลที่ต้องมีการใช้งานตำแหน่งบน หน่วยความจำพร้อมกัน ก็จะทำให้การ stall การอ่านให้ทำได้หลังจากการเขียนเสร็จสิ้นเรียบร้อยแล้ว ซึ่งวิธีนี้จะมี ปัญหาตรงที่ทำให้เกิด สถานะการหยุดชะงักของข้อมูลทำให้ประสิทธิภาพของ ของไปป์ไลน์ลดลงเป็นอย่างมาก

วิธีแก้ปัญหาอีกวิธีที่นิยมนำมาใช้กันคือ ทำการสร้างฮาร์ดแวร์ทำหน้าที่พิเศษขึ้นมารองรับการทำงาน โดยเฉพาะ การแยกการทำงานของ ช่วงสัญญาณในการอ่านและเขียน ข้อมูลลงบนหน่วยความจำ เช่น กำหนดให้ เขียนข้อมูลได้ที่ช่วงขอบขาขึ้น และอ่านข้อมูลได้ที่ช่วงขอบขาลง เป็นต้น การใช้ dual-port memory เพื่อให้ สามารถการอ่านและเขียนข้อมูลบนหน่วยความจำได้พร้อมกัน การทำ duplicate register เป็นการสร้างรีจิสเตอร์ ขึ้นมาอีกชุดหนึ่ง เพื่อให้สามารถเข้าถึงข้อมูลได้เร็วกว่าการเข้าถึงข้อมูลนั้น โดยตรงจากหน่วยความจำในกรณีของ เข้าถึงหน่วยความจำโดยอ้อมนั้นจะใช้ข้อมูลที่ชี้โดยตัวแปรค่าหนึ่ง ซึ่งต้องมีการอ่านข้อมูลจากหน่วยความจำถึง สองครั้ง การทำการ pre-calculate ในวงจร fetch ที่มีการกระโดด นั้นการรอให้มีการคำนวณว่ามีกระโดดขึ้น หรือไม่ ไปที่ตำแหน่งใดนั้น ใช้เวลามากและทำให้เกิดการละทิ้งคำสั่งที่ทำมา ซึ่งเกี่ยวข้องกับปัญหาด้านการควบคุม ด้วย ซึ่งการทำ pre-calculate นั้นจะช่วยคำนวณว่าจะเกิดการกระโดดไป ณ ตำแหน่งใด ซึ่งจะช่วยให้เกิดการ กระโดดได้เร็วยิ่งขึ้น